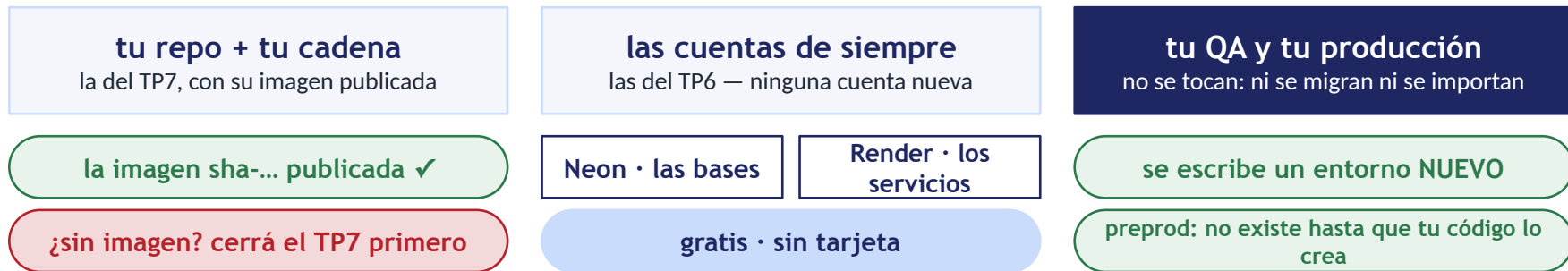


Ingeniería del Software 3

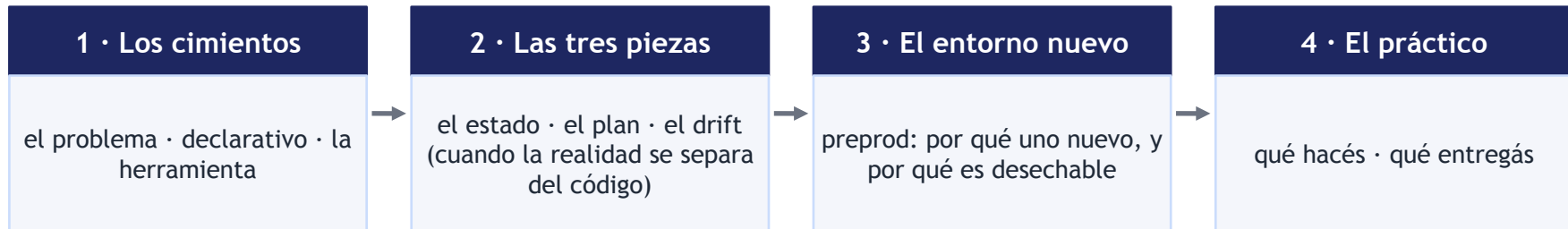
Clase 8 · Infraestructura como Código

UCC · Ingeniería en Sistemas · 2026 · Ing. Ariel Schwindt

Lo que vas a ver en este video

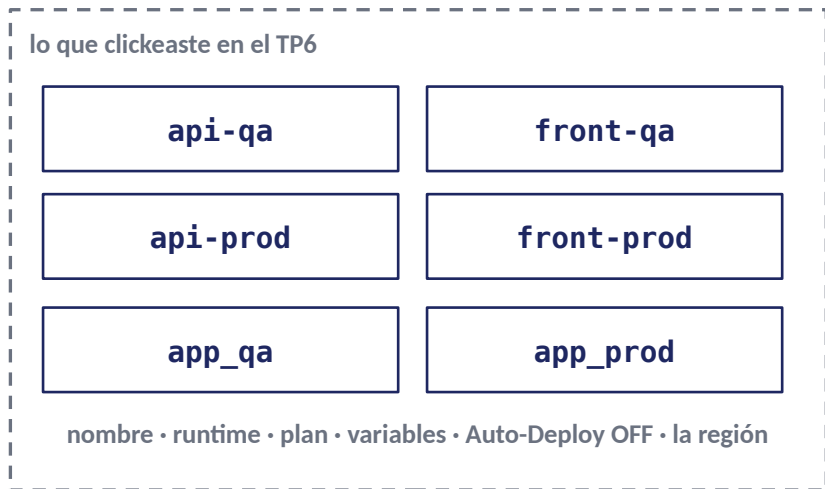


El orden del video



¿tus entornos corren en compose y no en la nube? El plan C de la guía es el MISMO práctico, corriendo contra tu Docker local

La pregunta: ¿lo podrías recrear mañana?



¿lo recreas IDÉNTICO mañana?

¿y tu compañero, sin mirar tu pantalla?

¿y en seis meses, cuando nadie se acuerde?

¿dónde está escrito?

en la cabeza del que lo hizo — y en clicks que nadie registró

no es que nadie quiera escribirlo

es que no hay ningún archivo que una máquina pueda ejecutar

Eso se llama **infraestructura artesanal**, y tiene nombre propio en la industria

Infraestructura artesanal: cuatro síntomas que vas a reconocer

snowflake servers

servidores únicos e irrepetibles como copos de nieve: se configuraron a mano, con parches, y hoy nadie sabe reconstruirlos

«en QA anda y en PROD no»

los dos entornos divergieron a fuerza de clicks — y nadie tiene la lista de en qué difieren

el README que miente

el del escenario del TP: el integrante nuevo tardó dos días porque la documentación estaba tres pasos atrás

el cambio que no se puede revisar

con el código lees el diff antes del merge; de un cambio de infra te enterás por el incidente

el más caro de los cuatro

es el único que no se arregla con más disciplina: no hay nada que revisar, porque el cambio no está escrito en ningún lado antes de ocurrir

Infraestructura como Código

describirla en archivos versionados que una herramienta materializa — la misma salida de todo el semestre

Con eso: se revisa en PRs • tiene historial • se recrea idéntica • deja de vivir en la cabeza de nadie

La quinta aparición del patrón – anunciada en la Clase 1



Lo que gana un equipo cuando todo lo importante vive en archivos versionados

se revisa ANTES de que pase

hay historial: quién y cuándo

se reconstruye idéntico

no vive en la cabeza de nadie

Queda UNA pieza del recorrido de la Clase 1: operar y observar lo desplegado – es el TP9, la clase que viene

Cuando en una entrevista te pregunten qué es DevOps en la práctica, esta fila es una respuesta con la que no fallás

Imperativo: el CÓMO, paso a paso

un script: la secuencia de acciones

1 creá el proyecto

2 creá la base adentro

3 creá el servicio conectado

¿te suena?

docker network create con el mismo nombre, dos veces: «already exists». Lo chocaste en el TP2

1ª corrida

todo bien: nada existía, todo se creó

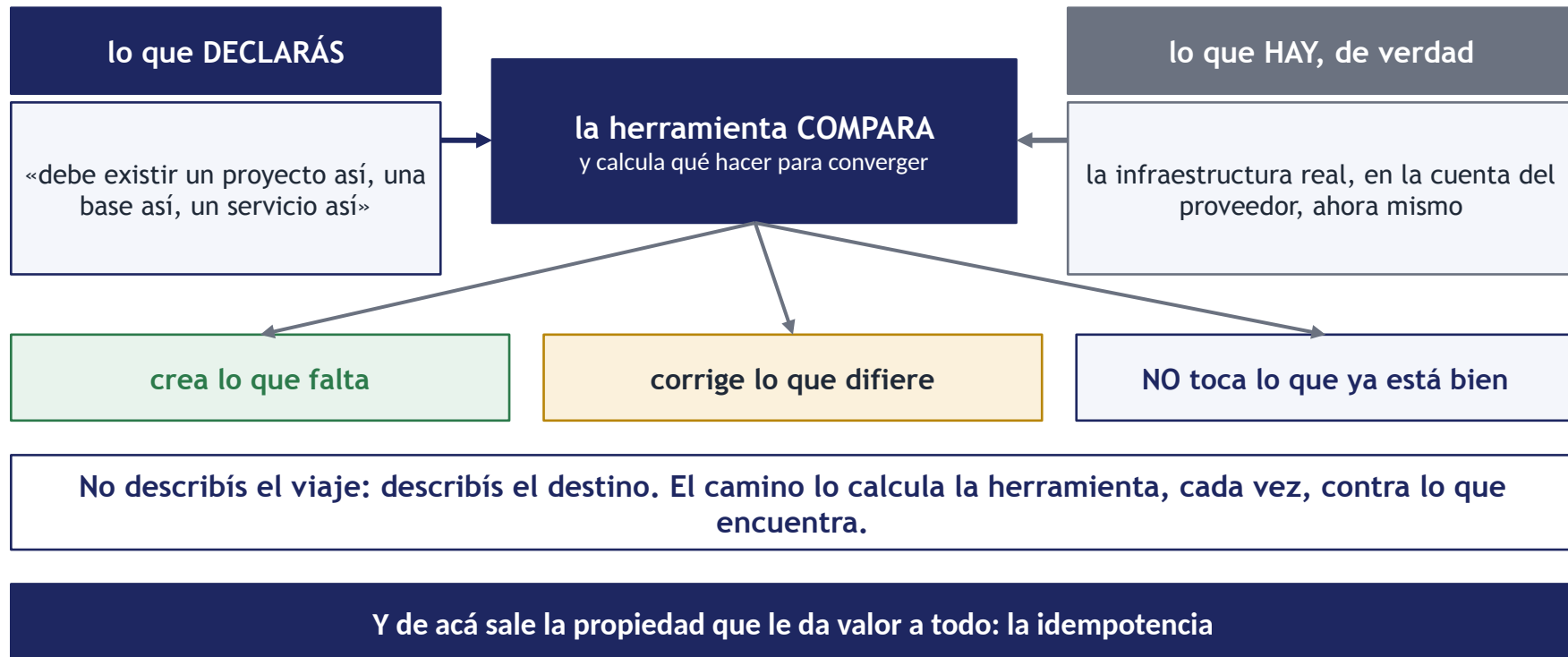
2ª corrida

falla («ya existe») – o peor: duplica
el script describe ACCIONES, no un destino

¿y si la primera quedó a medias?
el script no tiene idea de desde dónde retomar

Un script de infraestructura sólo es seguro la primera vez, sobre una cuenta vacía

Declarativo: el QUÉ, y la herramienta calcula el camino



Idempotencia: aplicar N veces es igual que aplicar una



Por qué importa tanto

1

tu configuración es SEGURA de re-aplicar

¿quedó bien? ¿se aplicó todo? Ante la duda, aplicás de nuevo — sin miedo a romper ni a duplicar

2

y permite que algún día lo aplique una máquina

aplicar sola sólo es viable si casi siempre no hace absolutamente nada

el script imperativo, segunda vuelta
falla «ya existe», o te deja dos proyectos idénticos

el declarativo, segunda vuelta
«No changes. Your infrastructure matches the configuration.»

Aplicar y que no pase nada no es una corrida desperdiciada: es la prueba de que la realidad coincide con el código

La herramienta: Terraform, y las que hacen lo mismo



No es la única

OpenTofu	la misma herramienta, en versión de código abierto	mismos archivos
Bicep	la de Microsoft	sólo Azure
CloudFormation	la de Amazon	sólo Amazon
Pulumi	lo mismo, escrito en un lenguaje de programación	varias nubes

Todas trabajan igual: declarás • te muestran qué va a pasar • aplican — acá usamos Terraform; con Azure, Bicep (§4 de la guía)

Un archivo .tf de cerca: la forma, y los cuatro comandos

lo que YA escribiste • compose.yml

```
services:
  db:
    image: postgres:16
    ports: ["5432:5432"]
```

lo mismo, en HCL • main.tf

```
resource "neon_project" "preprod" {
  name                       = "miapp-preprod"
  history_retention_seconds = 21600
}
```

La forma de TODO bloque de Terraform: la vas a reconocer en cada archivo de la guía

resource	"neon_project"	"preprod"	{ ... }
«esto tiene que existir»	el TIPO — lo pone el provider, no lo inventás vos	el NOMBRE — se lo ponés vos, para referenciarlo	los atributos: clave = valor

Los dos nombres entre comillas, juntos, son la DIRECCIÓN del recurso — `neon_project.preprod` —: con ése lo busca Terraform en el estado, toda su vida

Y los cuatro comandos, en el orden en que los vas a correr



El estado: la memoria de la herramienta



1

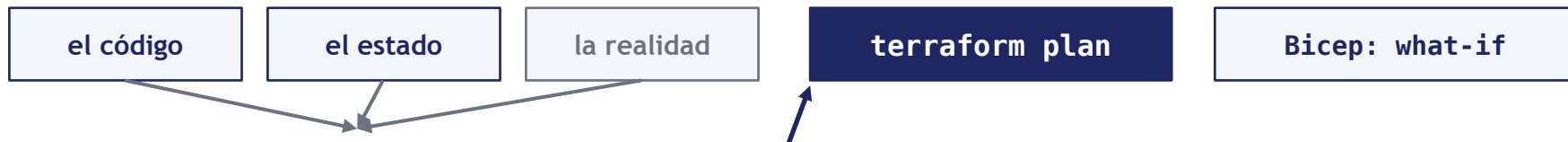
Qué guarda: un JSON que mapea cada recurso que declaraste con el recurso real que lo cumple – su id, sus atributos, de quién depende

2

Por qué hace falta: sin el puente no hay comparación, y sin comparación no hay plan. Borrá el estado y la herramienta cree que no existe nada de lo tuyo

Y adentro hay secretos: por eso el estado NUNCA se commitea

plan: el diff de la infraestructura, antes de tocarla



```
# neon_project.preprod will be created
+ resource "neon_project" "preprod" { name = "miapp-preprod" ... }
~ resource "neon_database" "preprod" { owner_name: "a" -> "b" }
- resource "render_web_service" "viejo" { ... } # will be DESTROYED
```

Plan: 1 to add, 1 to change, 1 to destroy.

+ crea

~ cambia en el lugar

- DESTRUYE

La última línea es la que se lee primero en la vida real: «1 to add, 1 to change, 1 to destroy». El día que diga «to destroy» y no lo esperabas, frenás.

La regla profesional: nunca apliques lo que no planeaste — y desconfiá de todo plan que destruya algo que no esperabas destruir

Drift: cuando la realidad se separa del código



```
$ terraform plan          # después de borrar la base a mano en la consola de Neon  
# neon_database.preprod will be created  
Plan: 1 to add, 0 to change, 0 to destroy. ← medido por la cátedra, 21-09
```

el drift no se prohíbe: se DETECTA

no hace falta que nadie avise: el próximo plan compara contra la realidad y muestra la diferencia

la reparación tiene DIRECCIÓN

la realidad se corrige hacia el código. ¿El cambio manual era legítimo? Entonces se trae AL código, por PR

Y el límite: Terraform vigila lo que DECLARASTE, no todo lo que existe — un plan limpio no dice «no hay nada raro en tu cuenta»

El entorno que falta: preprod



qué es un preprod, en la industria

un entorno lo más parecido posible a producción, donde se prueba la release antes de promoverla. También se lo llama staging, pre-producción o UAT

por qué existe

porque QA suele estar sucio y compartido —cualquiera lo pisa— y producción no se usa para probar. Hace falta un tercero, limpio

Y la diferencia con los otros dos: preprod no existe hasta que tu código lo crea, y desaparece cuando ya no lo necesitás

Al terminar este práctico vas a tener, en el mismo dashboard, entornos que existen porque alguien los clickeó y un entorno que existe porque está escrito. Uno lo recreás con un comando; los otros, no.

Y algo que vas a ver en el código: en preprod la cadena de conexión y la dirección del backend se DERIVAN de los otros recursos — no se copian a mano, como en el TP6

El límite del free tier, y la salida que este práctico enseña

El límite, con números: 750 horas por mes, del WORKSPACE y COMPARTIDAS entre tus seis servicios
si te pasás, Render suspende TODOS tus servicios gratuitos hasta el mes siguiente — tu producción incluida



y otra vez, cuantas veces haga falta

la regla del resto del semestre
cuando termines una sesión de trabajo, corré terraform destroy: es lo que hace que seis servicios entren en la cuota de cuatro

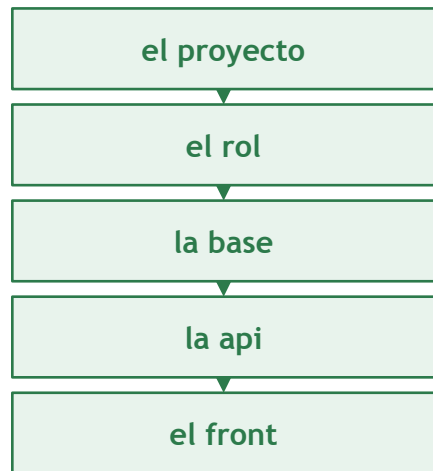
y no perdés nada
destruir preprod no es perder trabajo: el trabajo está en el repositorio

¿Podés permitirte tener un entorno más? Sí, si sabés hacerlo desaparecer

Y por eso preprod NO tiene que estar vivo el día de la defensa: tiene que poder NACER

El ciclo de vida: el orden lo decidió el grafo, no vos

apply • el orden de creación



verificá en las DOS consolas

nadie escribió ese orden

sale de las referencias que escribiste: el front nombra a la api, la api nombra a la base

eso es el grafo de dependencias

el orden EN EL ARCHIVO no importa; el grafo sí

y después: apply, y renace

el health responde — y la URL la lees del output, no de tu marcador

⚠ el PLAN del destroy va alfabético: el orden se ve al CONFIRMAR, en las líneas de «Destroying...»

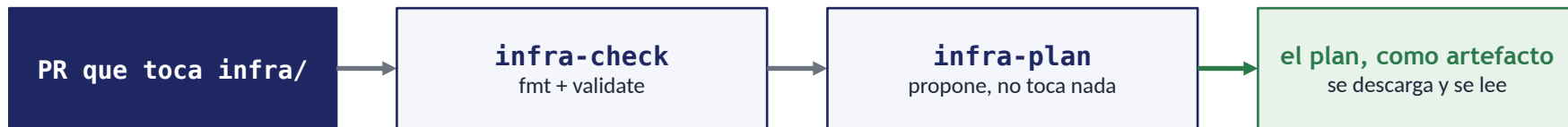
destroy • el grafo, al revés



y que QA y producción sigan intactos

«¿Podrías recrear tu infraestructura idéntica mañana?» — con IaC, la respuesta es un comando y unos segundos

Un paso más, opcional: el plan en cada Pull Request



Qué hace

cada Pull Request que toca la carpeta infra calcula el plan solo y lo deja adjunto, para leerlo antes de aprobar

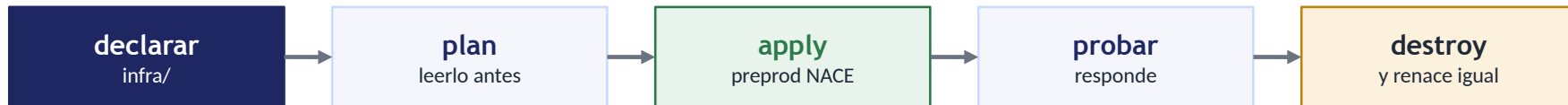
En este práctico es OPCIONAL

no se pide para aprobar · si lo querés hacer, está paso a paso en la sección 3.6 de la guía

Así se revisa la infraestructura en un equipo: el cambio se muestra antes de existir

TP8 — tres tareas, sobre un entorno que no existe

Escenario: alguien nuevo tardó dos días con un README que mentía • y esta semana hace falta un entorno más



- 1 El entorno preprod**
declarado y creado desde cero, con la imagen del TP7 • QA y producción NO se tocan • ninguna credencial en el código
- 2 El ciclo de vida**
destroy, y apply que lo devuelve idéntico • es lo que vas a hacer en la defensa
- 3 Subirlo, y la release v8.0.0**
infra/ a main por un pull request, sin el estado • y la release sobre ese commit

Práctica recomendada — no se entrega
idempotencia • el estado por dentro • drift • prevent_destroy. De ahí salen las preguntas de la defensa

Opcional
el plan en cada Pull Request (§3.6)

TP8 – Entregables

formulario

sólo la URL de tu repositorio, con la carpeta infra/ y la release v8.0.0 adentro

decisiones.md

tres cosas: qué riel y qué versiones usaste · qué problemas tuviste · qué hiciste con IA

no se pega nada

ni capturas, ni salidas de comandos, ni evidencias.md

En la defensa, en vivo: creás tu preprod

terraform apply · mostrás que la api y el front responden · terraform destroy — traelo ensayado

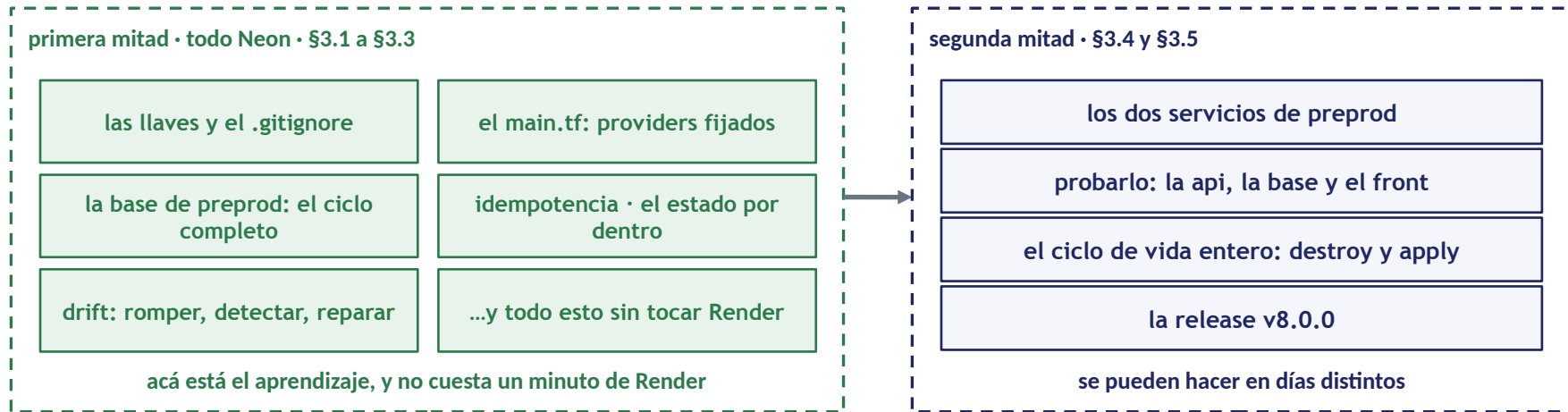
llegá con preprod DESTRUIDO: tiene que poder NACER

lo que SÍ sigue vivo: tus QA y tu producción

25% técnico

75% defensa

Por dónde empezar: la primera mitad no gasta un minuto de cuota



Empezá por la primera mitad: se hace entera sin gastar un minuto de la cuota de Render

Cierre — tu infraestructura ya no vive en clicks

1

TP8: preprod declarado, creado desde tu código, y capaz de morir y volver

El ciclo de vida, de punta a punta: destroy y apply, con QA y producción intactos

2

Practicá LEER un plan en voz alta

Qué se crea, qué cambia, qué se destruye — y cuál de esas líneas te daría miedo

3

Próxima clase: DevSecOps y observabilidad

Operar lo que ya está desplegado: seguridad continua, observabilidad y la cultura del incidente — más el secreto bloqueado antes del push